

Flujo del patrón MVC

Models

El **Modelo (Model)** es la capa encargada de gestionar los datos de la aplicación. Su principal responsabilidad es interactuar con la fuente de datos, independientemente de dónde se almacene la información.

Entre sus funciones se encuentran:

- Leer datos.
- Guardar datos.
- Buscar registros específicos.
- Actualizar información existente.
- Eliminar registros.

Una característica importante del patrón MVC es que el controlador no conoce la forma en que los datos son almacenados. En lugar de ello, solicita al modelo la información que necesita y este se encarga de realizar las operaciones correspondientes.

En este proyecto únicamente existe un modelo denominado **TurismoModelo.php**, el cual administra el archivo **lugares.json**. Este modelo contiene los métodos necesarios para obtener todos los lugares turísticos, buscar un lugar por su identificador, guardar nuevos registros, actualizar la información existente, eliminar lugares y generar automáticamente el siguiente identificador disponible.

Controllers

El **Controlador (Controller)** actúa como intermediario entre el Modelo y la Vista. Es el encargado de recibir las solicitudes realizadas por el usuario, decidir qué acción ejecutar, solicitar información al modelo cuando sea necesario y finalmente devolver una respuesta al navegador.

Sus principales responsabilidades son:

- Recibir las peticiones del usuario.
- Validar la información recibida.
- Solicitar datos al modelo.

- Enviar información a las vistas.
- Devolver la respuesta correspondiente.

Siguiendo el patrón MVC, el controlador no debe encargarse directamente del acceso a los datos ni de la presentación visual de la aplicación, ya que esas responsabilidades pertenecen al modelo y a la vista, respectivamente.

El proyecto cuenta con dos controladores:

1. ContactoController.php

Este controlador administra el flujo del formulario de contacto. Se encarga de mostrar el formulario al usuario, recibir la información enviada, validar los datos ingresados y almacenar las solicitudes en el archivo **contactos.json**. Una vez completado el proceso, redirige nuevamente al formulario mostrando un mensaje de confirmación. Su función principal es actuar como intermediario entre la vista del formulario y los datos almacenados.

2. TurismoController.php

Este controlador administra todas las operaciones relacionadas con el catálogo turístico. Permite listar los lugares disponibles, mostrar la información detallada de cada destino, crear nuevos registros, editar la información existente y eliminar lugares turísticos.

A diferencia del modelo, el controlador no accede directamente al archivo JSON. En su lugar, delega todas las operaciones relacionadas con los datos al **TurismoModelo**, el cual es el encargado de leer, guardar, buscar y eliminar la información. De esta forma, el controlador se limita a coordinar el flujo de la aplicación, validar los datos recibidos y enviar la información correspondiente a las vistas.

Views

Las **Vistas (Views)** representan la interfaz gráfica de la aplicación y son las encargadas de presentar la información al usuario.

Su única responsabilidad consiste en mostrar los datos enviados por el controlador mediante plantillas Blade. Las vistas no contienen la lógica del negocio ni realizan operaciones sobre los datos; únicamente presentan la información y permiten la interacción del usuario mediante formularios, enlaces y botones.

En este proyecto las vistas implementadas son:

- **index.blade.php:** muestra el catálogo de lugares turísticos disponibles.
- **show.blade.php:** presenta la información detallada de un lugar turístico.
- **create.blade.php:** contiene el formulario para registrar un nuevo lugar turístico.
- **edit.blade.php:** permite modificar la información de un lugar existente.
- **contacto/index.blade.php:** muestra el formulario de contacto para que los usuarios puedan solicitar información adicional.

Archivos JSON

Como fuente de datos, el proyecto utiliza archivos **JSON** ubicados en la carpeta **app/data**, los cuales funcionan como una base de datos local.

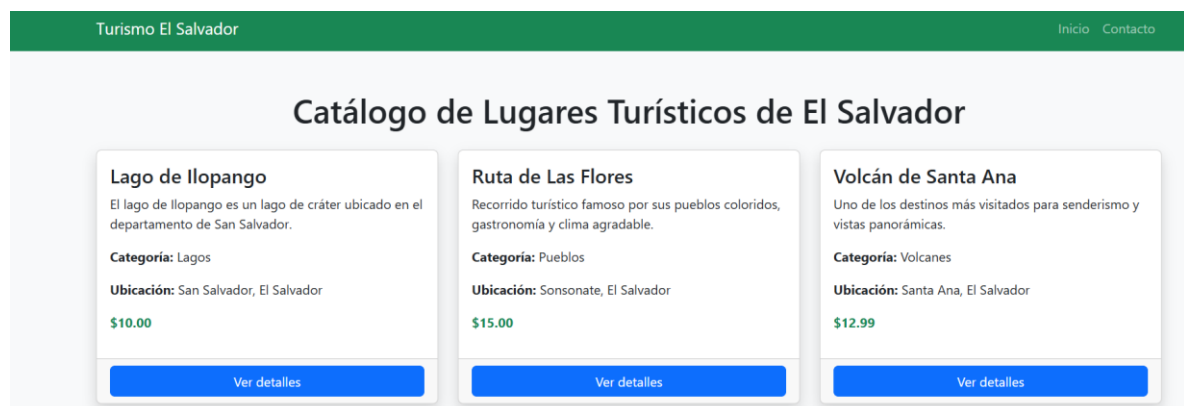
Los archivos utilizados son:

- **lugares.json**, donde se almacena toda la información de los destinos turísticos disponibles.
- **contactos.json**, donde se registran las solicitudes enviadas mediante el formulario de contacto.

Esta estrategia permitió implementar el funcionamiento del patrón MVC sin utilizar un sistema gestor de bases de datos, facilitando el manejo de la información mediante archivos estructurados en formato JSON.

Funcionamiento del sistema

Página de inicio (<http://127.0.0.1:8000/turismo>)



Ver detalles: se accede al darle click al botón azul de la imagen anterior, se puede acceder también siguiendo la ruta (<http://127.0.0.1:8000/turismo/id>)

Turismo El Salvador

[Inicio](#) [Contacto](#)

Lago de Ilopango

Descripción:
El lago de Ilopango es un lago de cráter ubicado en el departamento de San Salvador.

Categoría: Lagos

Ubicación: San Salvador, El Salvador

Precio: \$10.00

Regresar

Crear lugar: esta parte son extras para hacer pruebas no se pueden acceder de manera normal (<http://127.0.0.1:8000/turismo/create>)

Agregar Nuevo Lugar Turístico

Título

Descripción

Categoría

Precio

Ubicación

Guardar Lugar

Cancelar

Editar lugar: otra de las opciones implementadas para hacer pruebas, no se usa de manera normal y no forma parte de los requisitos mencionados se accede por la ruta (<http://127.0.0.1:8000/turismo/id/edit>)

Editar Lugar Turístico

Título

Lago de Ilopango

Descripción

El lago de Ilopango es un lago de cráter ubicado en el departamento de San Salvador.

Categoría

Lagos

Precio

10

Ubicación

San Salvador, El Salvador

[Actualizar Lugar](#) [Cancelar](#)

Contacto: en esta pagina se llena un formulario para realizar consultas sobre algún sitio turístico de le haya parecido interesante, la información se guardará en un json (**app/data**)

Turismo El Salvador [Inicio](#) [Contacto](#)

Formulario de Contacto

Nombre

Correo electrónico

Mensaje

[Enviar solicitud](#)